Learning Interpretable Scheduling Policies for Online Multi-Robot Task Allocation via Inductive Logic Programming

Mohammad Ibrahim Memon[1], **Ujjwal Patil**[1], **Iman Awaad**[1,2], **Youssef Mahmoud Youssef**[1]
[1]Hochschule Bonn-Rhein-Sieg
[2]German International University
{mohammad.memon, ujjwal.patil}@smail.inf.h-brs.de, {iman.awaad, youssef.youssef}@h-brs.de

Abstract

Scheduling incoming tasks across a fleet of mobile robots in a dynamic warehouse is an instance of online parallel machine scheduling with release times and machine eligibility constraints, where assignment decisions must account for resource constraints and dynamic system states. Conventional approaches rely on hand-crafted heuristics or market-based mechanisms that produce decisions without justification, making them difficult to audit, adapt, or trust in safety-critical deployments. We present a framework that learns a symbolic scheduling policy from robot execution logs using Inductive Logic Programming (ILP), producing human-readable Prolog rules that are both executable at runtime and interpretable by human operators. The learned policy assigns each arriving task to the idle robot above the critical battery threshold that is closest to the target location, and explicitly defers assignment when no eligible robot exists, a condition induced from data rather than programmed by hand. In a ROS 2 warehouse simulation across four fleet sizes, ILP delivers the best makespan when scheduling is hardest, remains statistically comparable to the best baselines at larger scales, and maintains competitive variance across all settings. Unlike all evaluated baselines, every allocation decision produced by our system is accompanied by a natural language justification derived directly from the matched rule conditions, providing an audit trail suited to multi-robot deployments where accountability and transparency are required.

1 Introduction

Modern warehouse operations increasingly rely on fleets of mobile robots to fulfil a continuous stream of incoming orders. Coordinating these robots effectively requires solving the task allocation problem at runtime: deciding which robot should handle each incoming order given the current state of the fleet. This decision must account for operational constraints including proximity to the goal, battery level, and current workload, and must be made quickly enough to keep pace with task arrival rates. The problem belongs to the broader class of online parallel machine scheduling with resource constraints (Graham 1969; Pinedo 2016), a setting that is known to be NP-hard in the general case and for which practical systems rely on heuristic or learned policies. A range of approaches exist including heuristic scoring, market-based and auction-based approaches, as well as optimisation methods (Gerkey and Matarić 2004; Korsah, Stentz, and Dias 2013). Auction-based approaches (e.g. (Gerkey and Matarić 2002)) and market-based (e.g. (Stentz and Dias 1999)) approaches differ in that auction-based approaches use estimated costs to make bids while market-based approaches use costs as well as benefits. Moreover, market-based approaches allow tasks to be re-assigned while there is no possibility of reassignment in auction-based approaches (as is also the case in this work) (Yan, Jouandeau, and Cherif 2011).

These approaches perform well in practice, but they share a fundamental limitation: they produce decisions without explanations. When a robot is selected, the system can report a score or a bid value, but it cannot say why that robot was the right choice in terms a human operator can interpret or verify. When an allocation fails or produces a suboptimal outcome, there is no symbolic trace to inspect. This lack of transparency becomes a practical concern as multi-robot deployments move into safety-critical settings where accountability and auditability of robot behaviour are required (Chakraborti, Sreedharan, and Kambhampati 2020).

A second limitation of conventional approaches is their lack of flexibility. Hand-crafted heuristics encode the designer's assumptions about what good allocation looks like. When those assumptions no longer hold, the rules must be rewritten and the system redeployed. This is a practical obstacle in warehouse environments where fleet composition, task distributions, and operational constraints evolve over time.

In this paper we address both limitations by framing task allocation as a policy induction problem. Rather than designing allocation rules by hand, we learn them from execution traces using Inductive Logic Programming (ILP) (Cropper and Dumančić 2022). The learned policy is represented as a set of Prolog clauses that are human-readable, auditable, and executable at runtime. The system also learns when allocation should be deferred: if no robot meets the conditions for safe assignment, the task is queued rather than force-assigned to a robot that is busy or battery-critical. This deferral behaviour corresponds to a learned precondition on action executability, connecting our approach directly to concepts from symbolic planning. The system is implemented on ROS 2 using the Popper ILP engine (Cropper and Morel

Copyright © 2026, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

2021) and evaluated in a simulated warehouse environment against five baselines spanning the main families of conventional Multi-Robot Task Allocation (MRTA) (see (Yan, Jouandeau, and Cherif 2013) for a thorough survey of approaches) and scheduling approaches.

MRTA has been studied extensively as an instance of combinatorial optimisation (Gerkey and Matarić 2004; Korsah, Stentz, and Dias 2013), with auction-based (Gerkey and Matarić 2002) and market-based (Stentz and Dias 1999) approaches representing the dominant practical paradigms. Scheduling-theoretic foundations for parallel machine problems are established in (Graham 1969; Graham et al. 1979). ILP as a learning framework is surveyed in (Cropper and Dumančić 2022), with Popper introduced in (Cropper and Morel 2021). To our knowledge, this is the first application of Popper-based ILP to online parallel machine scheduling with resource constraints.

The contributions of this paper are as follows:

1. We present an ILP-based MRTA framework that induces a symbolic, human-readable allocation policy from robot execution logs, formalised as an instance of online parallel machine scheduling with resource constraints.
2. We demonstrate that the learned policy includes an explicit deferral rule, acquired from data, that prevents force-assignment to battery-critical robots under high task load.
3. We show that the ILP allocator achieves the best makespan at the fleet size where allocation decisions are hardest, remains competitive with the best baselines at larger fleet sizes, and produces consistently low makespan variance across all conditions, in a comparative simulation study involving five baselines across four fleet sizes.
4. We show that every allocation decision produced by the system is accompanied by a human-readable justification derived from the matched rule conditions, providing an audit trail that is absent from all evaluated baselines.

The remainder of the paper is structured as follows. Section 2 formalises the scheduling problem. Section 3 describes the system architecture. Section 4 presents the ILP learning pipeline and learned policy. Section 5 reports the experimental evaluation. Section 6 discusses limitations and future work, and Section 7 concludes.

2 Problem Formulation

We consider a fleet of m mobile robots operating in a warehouse environment. Robots act as identical parallel machines, each capable of handling one order at a time. A stream of orders arrives online with release times r_j, where each order j specifies a target shelf location. Orders must be assigned to robots as they arrive; the full sequence of orders is not known in advance.

Each robot r is characterised at any point in time by three state variables: its current location, its battery level $b_r \in [0, 1]$, and its availability status. A robot is eligible for assignment only if it is currently idle and its battery level exceeds a critical threshold b_{min}. This eligibility constraint reflects the physical requirement that a robot must have sufficient charge to complete a round trip to the target shelf and return to the drop-off point.

The processing time of order j assigned to any robot is:

$$p_j = \frac{2 \cdot d(\text{drop_off},\ \text{shelf}_j)}{v} \tag{1}$$

where d is the Euclidean distance between the drop-off point and the target shelf location, and v is the robot travel speed. Processing time is job-dependent but not sequence-dependent, because all robots begin each assignment from the same fixed drop-off location and return to it upon completion.

The optimality criterion is to minimise the makespan (when the final job completes):

$$\text{minimise } C_{max} \tag{2}$$

Using standard three-field scheduling notation (Graham et al. 1979), this problem is denoted:

$$Pm \mid r_j,\ \text{eligible} \mid C_{max} \tag{3}$$

When no eligible robot is available at the time of order arrival, the order is deferred and re-evaluated at the next allocation cycle rather than force-assigned to a battery-critical robot.

2.1 Hardness and Motivation for a Learned Policy

The offline variant of this problem, where all orders are known in advance and eligibility constraints are relaxed, reduces to the classical $Pm \mid\mid C_{max}$ problem, which is NP-hard for $m \geq 2$ (Garey and Johnson 1979). The Longest Processing Time (LPT) rule provides a $\frac{4}{3}$ approximation for the offline case (Graham 1969), but it is not directly applicable here for two reasons. First, orders arrive online and must be assigned upon arrival without knowledge of future orders. Second, the eligibility constraint introduces a feasibility condition on assignment that the classical LPT formulation does not model. While an online LPT algorithm exists, it does not guarantee the 4/3-approximation ratio of offline LPT (Chen and Vestjens 1997).

These properties motivate a learned policy rather than a derived approximation. Rather than computing an analytic scheduling rule, we learn one from execution traces, allowing the policy to capture the eligibility constraint and online arrival structure implicitly through the training data.

2.2 Scope and Assumptions

This work makes the following assumptions to define the scope of the problem addressed:

- Orders are atomic: each order is assigned to exactly one robot and executed without preemption.
- Robots are homogeneous: all robots travel at the same speed v and are capable of handling any order.
- Task decomposition is performed upstream: orders arrive as atomic assignments. The problem of decomposing complex tasks into sub-orders is beyond the scope of this work.

- A single drop-off point serves as the origin for all order assignments.
- Allocation decisions are made centrally based on observed fleet state (No robot-to-robot communication is necessary).

Extensions to heterogeneous fleets, multi-robot tasks, and task decomposition are directions for future work.

3 System Architecture

The system is implemented on ROS 2 Humble and consists of three core nodes. The *Task Manager Node* receives incoming orders, maintains a task queue, and invokes the active allocation strategy to select a robot for each order. When the ILP allocator is active, it queries a live Prolog runtime via a Python-Prolog bridge to evaluate the learned scheduling rules against the current fleet state. The *Fleet Manager Node* maintains the ground truth state of the robot fleet, tracking each robot's current location, battery level, workload, and availability. It dispatches accepted assignments to the navigation stack via Nav2 waypoints, appending a return-to-drop-off waypoint after each shelf visit so that robots are always available from a known location for the next assignment. The *Shared Memory Node* provides static environment data to other nodes via ROS 2 services, including shelf locations and the drop-off pose.

Figure 1 provides an overview of the full system architecture, including the execution layer, shared state layer, allocation layer, and the offline logging and learning pipeline.

The Task Manager and Fleet Manager communicate through ROS 2 service calls. Fleet state is refreshed at 2 Hz.

3.1 Headless Simulation for Reproducible Experiments

All experiments are conducted using a headless simulation mode that replaces the full Gazebo environment with a mock fleet manager. The mock fleet manager simulates robot movement, battery drain, and recharge without requiring a graphical simulation backend, enabling fast and reproducible experimental runs. Robot travel is modelled at $v = 2.0$ m/s with battery drain of 0.7% per metre and recharge at 5.0% per second. Orders arrive at a fixed rate of 1.0 orders per second, generated from a seeded random process to ensure identical task sequences across all evaluated strategies.

3.2 Allocation Strategies

Six allocation strategies are implemented as interchangeable components within the Task Manager. Each strategy implements the same interface, receiving the current fleet state and order details and returning a robot identifier, and an allocation decision. This design allows strategies to be switched between trials without modifying any other part of the system. The six strategies are summarised in Table 1.

Strategy	Type	Role
Random	Baseline	Performance floor
Score	Heuristic	Weighted distance and battery score
Heuristic	Heuristic	Three-tier priority rule, ILP teacher
Auction	Market-based	Minimum cost bid
Online LPT	Scheduling	Least processing time (Graham 1969)
ILP	Learned	Popper-induced Prolog rules (Cropper and Morel 2021)

Table 1: Allocation strategies evaluated in this work.

4 The ILP Scheduling Policy

This section describes how the scheduling policy is represented, learned, and executed. The central design choice is to learn a symbolic dispatch rule from execution traces rather than hand-code a fixed numeric priority function. The resulting policy is expressed as Prolog clauses that can be inspected directly, executed online, and translated into a human-readable explanation for each allocation decision.

4.1 Data Collection and Feature Engineering

Training data is collected at the level of an *allocation event*. Each event corresponds to one arriving job and contains one candidate row per robot at the instant the scheduler is queried. The logger records, for every candidate robot, whether it is idle, its current active workload, its battery level, its distance to the requested shelf, and the final dispatch decision. Events are then grouped by task identifier and timestamp before being converted into ILP training examples.

The representation is intentionally symbolic. Continuous state variables are discretised into a small vocabulary of relational facts shared by the logger, the offline data generator, and the runtime allocator. Battery is mapped to ordinal buckets (`low`, `medium`, `high`), distance to (`near`, `medium`, `far`), and active workload to (`zero`, `low`, `medium`, `high`). In addition, the system derives comparative predicates over the current candidate set, including whether a robot is the closest idle candidate, whether it has the lowest active workload, and whether it is above the critical battery threshold of 20%.

The most important derived predicates are those that already encode scheduling structure rather than raw geometry alone:

- `suitable_robot(Task,Robot)`: the robot is idle and above the critical battery threshold;
- `least_busy_suitable(Task,Robot)`: the robot is suitable and belongs to the minimum-workload tier;
- `closest_suitable_idle(Task,Robot)`: the robot is suitable and is tied for minimum distance;
- `closest_least_busy_suitable(Task,Robot)`: the robot is both in the minimum-workload suitable tier and closest within that tier;
- `no_suitable_robot(Task)`: no idle robot satisfies the battery eligibility constraint.

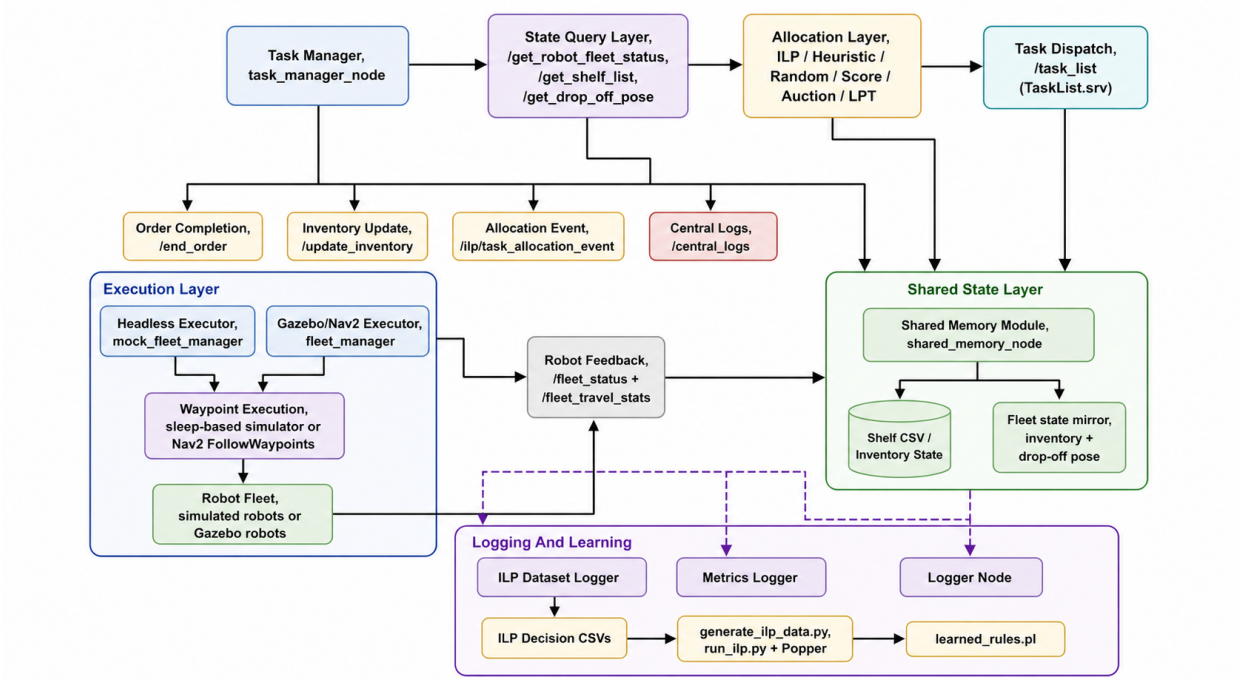


Figure 1: System architecture. Incoming orders are processed by the Task Manager, which queries the fleet state and invokes the active allocation strategy. Allocation events are logged offline and used to train the ILP policy via Popper, producing `learned_rules.pl` which is loaded at runtime by the ILP allocator.

This language makes the induced hypothesis interpretable by construction. Instead of forcing ILP to recover an opaque weighted score, it can search over predicates that already correspond to recognisable scheduling concepts such as eligibility, load balance, and local travel cost.

4.2 Teacher Policy and Training Example Construction

Training traces are generated by a symbolic teacher policy that applies a strict lexicographic priority order:

1. prefer eligible robots, meaning idle robots above the critical battery threshold;
2. among eligible robots, prefer the minimum active workload;
3. within that workload tier, prefer the shortest distance to the requested shelf;
4. use battery level and robot identifier only as deterministic tie-breakers.

This teacher is important for two reasons. First, it provides a stable supervision signal that is already close to the scheduling semantics we want the learned policy to express. Second, because the teacher applies an explicit symbolic priority rule, the resulting training examples are already aligned with the predicate vocabulary used by Popper, ensuring that the supervision signal is consistent with the hypothesis space the learner will search.

The offline script converts each logged event into Popper background knowledge and labelled examples. For the dispatch target, the selected robot becomes a positive instance of `assigned(Task,Robot)` and the remaining candidates become negative instances. For the deferral target, each event yields either a positive or negative instance of `defer(Task)` depending on whether the scheduler postponed the assignment.

4.3 Learning with Popper

We use Popper as the ILP engine (Cropper and Morel 2021). The learning problem is decomposed into two targets:

$$\texttt{assigned/2} \quad \text{and} \quad \texttt{defer/1}. \tag{4}$$

Learning these targets separately keeps the hypothesis space small and reflects the operational semantics of the scheduler: assigning a job and deferring a job are distinct decisions.

For `assigned/2`, the bias exposes 17 body predicates, including bucket predicates, availability predicates, and comparative predicates such as `least_busy_suitable/2` and `closest_least_busy_suitable/2`. For `defer/1`, the bias contains only four fleet-level predicates: `all_robots_busy/1`, `no_suitable_robot/1`, `has_idle_robot/1`, and `has_suitable_robot/1`. The search is capped at three rules, four variables, and five literals per clause. Each target is learned in a fresh subprocess to avoid state leakage across Popper runs.

The learned program deployed by the runtime allocator

is:

```
assigned(T,R) ←
    closest_least_busy_suitable(T,R)   (5)

defer(T) ← no_suitable_robot(T).   (6)
```

Rule 5 implements a greedy approximation of the scheduling objective in Equation (2), while Rule 6 enforces the machine eligibility constraint from Section 2 as a learned symbolic precondition.

4.4 Scheduling Interpretation of the Learned Rules

Equation 5 is more structured than a simple nearest-robot policy. It first enforces the machine eligibility constraint through `suitable_robot/2`, then restricts attention to the lowest-workload eligible tier, and only then minimises distance within that tier. In scheduling terms, the rule combines executability, load balance across parallel machines, and a local approximation of processing cost.

This structure is useful for two reasons. First, the learned policy can be stated in a single sentence: assign a job to the closest eligible idle robot among those with the smallest active workload. Second, each component of the rule maps to a term that a planning or scheduling reader can evaluate independently. If the policy performs poorly, one can ask whether the issue lies in the eligibility condition, the workload preference, or the distance preference, rather than trying to interpret a single opaque score.

Notably, Popper was not provided with an explicit scheduling algorithm, objective function, or scheduling-theoretic knowledge. It received only labelled allocation examples and relational background predicates. The resulting rule nevertheless recovers a scheduling-relevant priority structure: enforce eligibility, prefer the least-loaded suitable robots, and minimise distance within that tier. This structure is consistent with the classical least-loaded-machine scheduling rule adapted for machine eligibility constraints (Graham 1969).

4.5 The Deferral Rule as a Learned Precondition

Equation 6 is the second learned component and is conceptually important. Rather than treating deferral as an ad hoc exception, the system learns an explicit symbolic condition under which dispatch should not occur. In planning terms, this is a learned precondition on action executability: if no suitable robot exists, the correct action is to leave the job in the queue and wait for the state to change.

This learned precondition matters as it separates *infeasibility* from *poor optimisation*. Force-assigning to an ineligible robot merely to avoid a single-cycle queue delay would violate the machine eligibility constraint and risk task failure. The symbolic deferral clause makes that decision auditable and easy to revise if domain assumptions change.

4.6 Runtime Inference and Explanation Generation

At runtime, the allocator rebuilds the current symbolic state from the live fleet snapshot, asserts the resulting facts into SWI-Prolog through the Janus bridge, queries `assigned(current_task,Robot)` and `defer(current_task)`, and then retracts the temporary facts. The same symbolic vocabulary is used in training and execution, which avoids a common mismatch between an offline learned model and an online decision layer.

Each decision is accompanied by a natural-language explanation generated from the predicates satisfied by the winning candidate. For example, an assignment decision is logged as selecting a robot because it matched `closest_least_busy_suitable(Task,Robot)`, with explicit evidence that the robot was suitable, near the target shelf, and had zero active workload. When several robots satisfy the same learned rule, the runtime system applies a deterministic numeric tie-breaker and states this explicitly in the explanation.

Across evaluation runs, the ILP allocator produced 600 successful assignment decisions. Of these, 589 directly cited the learned rule in Equation 5, while the remaining 11 explicitly reported that multiple robots matched the symbolic rule and a tie-breaker was applied. No ILP decision in these runs invoked the heuristic fallback. Mean allocator runtime ranged from 5.75 ms at n=5 to 33.27 ms at n=15, which is small relative to job execution time and confirms that symbolic explanations can be delivered online without materially slowing dispatch.

Table 2 illustrates three representative allocation decisions drawn directly from the evaluation logs. In the first case, a single robot unambiguously satisfied the learned rule and was selected with a direct justification. In the second, multiple robots matched the rule and the system applied a deterministic tie-breaker, stating this explicitly in the logged explanation. In the third, every robot remained above the battery eligibility threshold, but all 10 robots were already busy, so the task was deferred, with the explanation identifying the specific queue-preserving constraint that prevented assignment. In all three cases, the explanation is derived mechanically from the matched predicates and defer rule, requiring no post-hoc interpretation.

5 Experimental Evaluation

We evaluate the learned symbolic scheduler against five baselines across four fleet sizes in the headless warehouse simulator. The goal is to assess whether the ILP policy is competitive with conventional scheduling approaches on standard efficiency metrics, and to characterise how its relative performance changes as the fleet transitions from a resource-constrained to an over-provisioned operating regime.

5.1 Experimental Setup

We evaluate six online allocation strategies in the headless warehouse simulator: Random, Score, Heuristic, Auction, ILP, and an online least-processing-time baseline denoted

Decision	State at Decision Time	Explanation Basis
Assignment	15 robots idle, all with zero workload and 100% battery. Robot R2 was the closest candidate at 4.21 m.	R2 matched `closest_least_busy_suitable(T,R)`. The explanation cites high battery, near distance, and zero workload.
Assignment with tie-break	R10 and R14 both matched the learned rule at 4.70 m. R10 was selected because it had higher battery (100% vs. 92.9%).	Multiple robots satisfied the symbolic rule. The deterministic battery tie-breaker selected R10 and was reported explicitly.
Deferral	All 10 robots were busy with workload 1. No idle candidate satisfied the assignment condition.	The task matched the learned deferral rule `defer(T) :- all_robots_busy(T)` and remained queued.

Table 2: Representative ILP allocation decisions and their explanation basis. Each explanation is generated from the symbolic predicates satisfied at decision time. Assignment requires idle status and battery above 20%.

`online_LPT`. Each trial contains 50 arriving jobs released at 1 Hz, and each strategy is evaluated on three trials for each fleet size. The fleet is homogeneous, robots move at 2.0 m/s, and a robot is considered ineligible for dispatch when its battery falls below the critical threshold of 20%.

The four fleet sizes span distinct operating regimes. The n=5 condition is clearly resource-constrained, n=10 is moderately provisioned, n=15 is lightly loaded, and n=20 is effectively over-provisioned for the 50-job horizon. This progression is useful because it tests whether the learned symbolic policy is only helpful under congestion or remains competitive after the queuing pressure disappears.

5.2 Evaluation Metrics

We report the standard scheduling metrics used throughout the paper:

- **Makespan**: total time from the first job start to the final job completion in a trial;
- **Throughput**: completed jobs per second of makespan;
- **Mean latency**: average job completion delay;
- **Success rate**: fraction of jobs completed successfully;
- **Allocator runtime**: wall-clock time required to make one dispatch decision;
- **Travel distance**: average Euclidean distance associated with an assignment;
- **Battery consumed**: average battery usage per assignment;
- **Deferral rate**: fraction of allocation events that leave the job queued.

All six strategies achieved a success rate of 1.0 in every fleet-size condition. The comparisons below are therefore driven by timing, queueing, and resource-usage differences rather than by outright task failure.

5.3 Baseline Strategies

Random. Random dispatch selects uniformly among currently idle robots. It provides a lower-bound baseline with no explicit scheduling structure.

Score. Score dispatch uses a weighted numeric priority function over workload, distance, and battery. It represents a generic hand-tuned scheduler that can trade off multiple features, but whose behavior is not directly interpretable as a symbolic rule.

Strategy	n=5	n=10	n=15	n=20
Random	162.09	83.03	**62.90**	63.02
Score	161.77	79.67	63.58	64.03
Heuristic	159.76	78.50	64.09	**61.85**
Auction	160.93	78.19	63.59	62.02
ILP	**159.38**	**77.59**	63.78	61.87
Online LPT	161.92	79.22	63.93	66.03

Table 3: Mean makespan in seconds over three trials. Lower is better. Best result in each fleet size is shown in bold.

Heuristic. Heuristic dispatch is the symbolic teacher policy used for ILP training. It applies a strict lexicographic ordering over eligibility, active workload, and distance, with deterministic tie-breaking.

Auction. Auction dispatch lets each robot submit a bid based on distance, battery, and workload, and allocates the job to the highest bidder. This baseline represents a market-style scheduler.

ILP. The ILP allocator executes the learned symbolic rules from Section 4. Its distinguishing feature is that each dispatch decision is both machine-executable and human-readable.

Online LPT. The final baseline is an online adaptation of the Longest Processing Time scheduling family. Because jobs arrive online, it does not sort the full job set in advance. Instead, it assigns each arriving job to the eligible robot with the smallest accumulated nominal processing load, implementing the least-loaded-machine rule at each decision point. This baseline connects the empirical evaluation to classical parallel machine scheduling theory (Graham 1969).

5.4 Results Across Fleet Sizes

Table 3 reports the mean makespan for all six strategies across the four fleet sizes, and Figure 2 visualizes the same scaling trend. ILP achieves the best mean makespan at n=5 and n=10, ranks fourth at n=15, and is effectively tied for first at n=20, where it trails Heuristic by only 0.023 s. Averaged over the four fleet sizes, ILP has the best overall makespan rank of 2.0, ahead of Auction (2.75), Heuristic (3.0), Score (4.0), Random (4.25), and Online LPT (5.0).

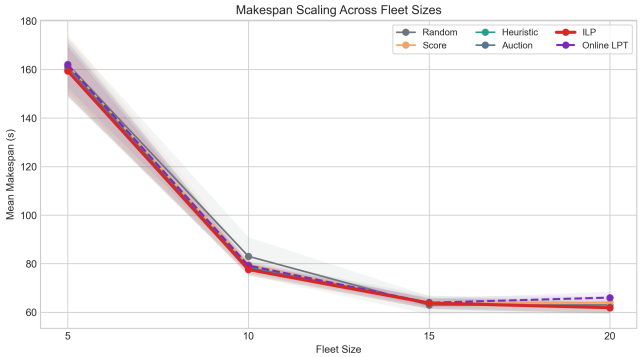


Figure 2: Mean makespan versus fleet size for all six online schedulers. The figure highlights two effects: the separation between strategies is largest in the congested regimes (n=5 and n=10), and the performance gap collapses as the fleet becomes over-provisioned.

Two patterns are immediately visible. First, the largest separation between strategies occurs under higher contention. At n=10, ILP improves on Heuristic by 0.91 s, on Auction by 0.60 s, and on Online LPT by 1.63 s. At n=5, where the system is heavily queue-constrained, ILP again yields the best makespan, though the margin over Heuristic is smaller at 0.38 s.

Second, the strategy spread compresses as the fleet becomes less constrained. At n=15, the entire makespan range from best to worst is only 1.19 s. At n=20, ILP, Heuristic, and Auction are all tightly clustered, with only 0.17 s separating first from third. This is the operating regime where explainability becomes most important: once the fleet is over-provisioned, the practical distinction between policies is no longer whether they complete the jobs, but how transparent and adaptable their dispatch logic is.

The same pattern appears in throughput and latency. ILP attains the highest throughput at n=5 and n=10, and is within 0.0003 jobs/s of the best method at n=20. Its mean latency is the best at n=10 (17.62 s), within 0.07 s of the best method at n=15, and within 0.05 s of the best method at n=20. The learned symbolic policy therefore remains competitive not only on end-of-horizon makespan, but also on average per-job responsiveness.

At n=15, Random attains the best mean makespan but ranks fifth on mean latency. Its makespan standard deviation of 3.79 s is the highest among all six strategies, suggesting that its advantage on this metric is attributable to variance across trials rather than systematic scheduling superiority. ILP, by contrast, achieves a lower makespan standard deviation of 2.30 s and ranks third on latency, reflecting more consistent performance across conditions.

5.5 Online LPT as a Scheduling-Theoretic Baseline

The Online LPT baseline connects the empirical comparison to classical parallel machine scheduling theory (Graham 1969), allowing the learned symbolic policy to be assessed against a principled scheduling heuristic in addition to the domain-specific approaches. Despite this theoretical grounding, Online LPT does not outperform the embodied dispatch policies in our experiments. Its mean makespan is worse than ILP at every fleet size: 2.55 s at n=5, 1.63 s at n=10, 0.15 s at n=15, and 4.16 s at n=20.

This outcome is consistent with the allocator design. Online LPT optimizes cumulative nominal processing load computed from a fixed drop-off-to-shelf model, rather than the robots' current geometric state in the workspace. That abstraction is well motivated from a scheduling standpoint, but it ignores some of the embodied information exploited by the symbolic policies, especially current proximity and instantaneous availability. This mismatch also appears in the resource-usage metrics: Online LPT consistently travels farther and consumes more battery per assignment than ILP, Heuristic, and Auction.

Online LPT is not without merit. At n=10, for example, it has the lowest makespan standard deviation (1.61 s), suggesting relatively consistent behavior across trials. However, its mean makespan and mean latency are still worse than ILP, Auction, and Heuristic in that condition.

Strategy	Runtime (ms)	Travel (m)	Battery (%)
Random	0.097	23.85	16.70
Score	0.101	22.53	15.77
Heuristic	0.095	21.96	15.37
Auction	0.098	22.02	15.41
ILP	16.257	21.98	15.38
Online LPT	0.087	24.02	16.81

Table 4: Aggregate secondary metrics across all runs. Runtime is averaged over all allocation events; travel distance and battery consumption are averaged over successful assignments.

5.6 Discussion

Table 4 reports aggregate resource-usage metrics across all experimental conditions. ILP achieves average travel distance and battery consumption per assignment that are nearly identical to Heuristic and Auction, confirming that the learned symbolic policy selects robots as efficiently as the best hand-coded baselines. Online LPT and Random consistently travel further and consume more battery, reflecting their lack of proximity awareness in the dispatch decision.

ILP's average travel distance (21.98 m) is almost identical to Heuristic (21.96 m) and Auction (22.02 m), and its average battery consumption per assignment (15.38%) is likewise almost identical to Heuristic (15.37%). By contrast, Online LPT averages 24.02 m of travel and 16.81% battery use per assignment, while Random averages 23.85 m and 16.70%. The learned symbolic policy therefore achieves its competitive makespan without paying an efficiency penalty in motion or energy.

The deferral behavior reinforces the same interpretation. For ILP, the event-level deferral rate falls from 58.2% at n=5 to 31.8% at n=10, then effectively disappears at n=15 (0.7%) and n=20 (0%). This is exactly the qualitative behavior one would expect from a learned executability condi-

tion in an online scheduling system: when the fleet is scarce, the scheduler must often wait; when the fleet is abundant, the deferral rule becomes almost inactive.

The main cost of ILP is allocator runtime. Mean decision time rises from 5.75 ms at n=5 to 18.52 ms at n=10, peaks at 33.27 ms at n=15, and is 20.96 ms at n=20. The other strategies remain near 0.1 ms. Even so, the absolute ILP runtime remains small relative to job execution times measured in seconds, so the symbolic overhead is operationally negligible in this setting.

The results support three conclusions. First, the learned symbolic policy is the best overall method by average makespan rank across fleet sizes, and remains competitive with the strongest baselines on latency and throughput at every condition tested. Second, its relative advantage is largest in congested regimes where eligibility-aware deferral and workload-sensitive dispatch have the most impact on queue management. Third, as the fleet becomes over-provisioned and performance gaps collapse, ILP retains practical value through a property none of the baselines provide: a human-readable justification for every dispatch decision, forming an audit trail that remains available regardless of operating regime.

Figure 3 summarises the ranking of all six strategies across makespan, latency, and throughput at each fleet size. The ILP column is consistently among the darkest across the efficiency metrics, confirming its competitive performance across operating regimes. The uniform dark colouring of the success rate rows reflects the 100% task completion achieved by all strategies in every condition.

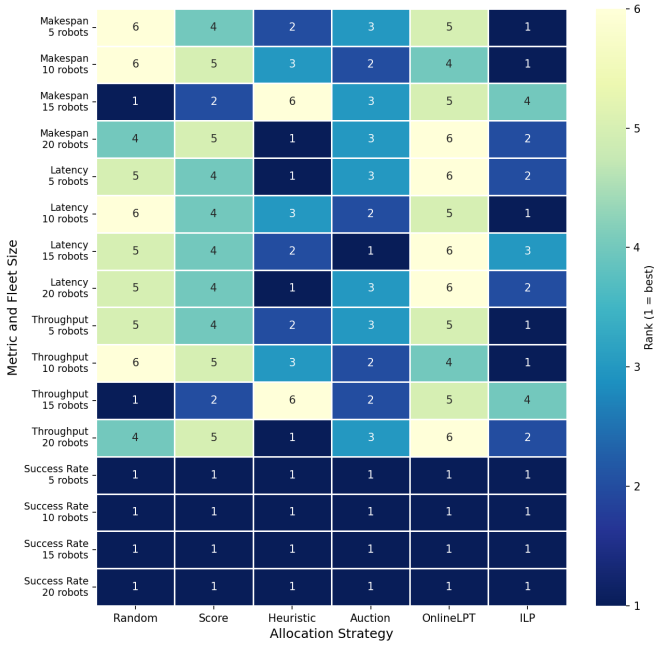


Figure 3: Strategy ranking across fleet sizes and metrics. Darker cells indicate better rank (1 = best). Success rate rows are uniformly dark, reflecting 100% task completion across all conditions. ILP ranks first or second on makespan and throughput in the congested regimes (n=5, n=10) and remains in the top three across most conditions.

6 Limitations and Future Work

The current evaluation has several limitations that inform future directions. Each fleet size was evaluated over three trials, which is sufficient to identify consistent trends but limits the statistical power of individual comparisons. Future work should replicate the study with a larger number of trials and report confidence intervals to strengthen the empirical claims.

The experimental setup assumes a homogeneous fleet of identical robots operating at the same speed with uniform battery dynamics. Real warehouse deployments involve heterogeneous fleets where robots differ in speed, payload capacity, and battery characteristics. Extending the ILP feature vocabulary and background knowledge to capture inter-robot differences is a natural next step.

The current system receives orders as atomic assignments and assumes task decomposition is performed upstream. Integrating task decomposition directly into the ILP learning pipeline, for example by learning preconditions over compound tasks, would bring the approach closer to full multi-robot task planning.

The experiments use a fixed deterministic order arrival rate of 1 Hz. Robustness to stochastic arrival patterns, such as Poisson-distributed inter-arrival times, has not been evaluated. Since the learned deferral rule is stateless with respect to arrival history, the policy is expected to generalise well, but this remains to be verified empirically.

ILP allocator runtime grows with fleet size, reaching 33 ms at n=15. While this is negligible relative to job execution times in the current setting, larger fleets may require optimisation of the Prolog fact assertion and retraction cycle to maintain sub-second decision latency.

Finally, the online retraining pipeline is implemented but was not evaluated in this study. Assessing whether the system can adapt its learned rules incrementally as fleet conditions change, without requiring a full offline retraining cycle, is an important direction for deployment in dynamic environments.

7 Conclusion

We presented a framework for learning interpretable scheduling policies for online multi-robot task allocation using Inductive Logic Programming. The learned policy, expressed as two Prolog clauses, assigns each arriving job to the closest idle eligible robot with the lowest active workload, and defers assignment when no eligible robot exists. This deferral condition is induced from data rather than programmed by hand, and corresponds to a learned precondition on action executability in planning terms.

Evaluated against five baselines across four fleet sizes, the ILP allocator achieves the best overall makespan rank, performs strongest in the congested regimes where allocation decisions are hardest, and matches the operational efficiency of the best hand-coded baselines on travel distance and battery consumption at all fleet sizes. Unlike every evaluated

baseline, it produces a human-readable justification for every dispatch decision, providing an audit trail suited to deployments where accountability is required.

Acknowledgments

We gratefully acknowledge the support of the Hochschule Bonn-Rhein-Sieg and the Bonn-Aachen International Center for Information Technology (b-it).

References

Chakraborti, T.; Sreedharan, S.; and Kambhampati, S. 2020. The emerging landscape of explainable ai planning and decision making. *arXiv preprint arXiv:2002.11697*.

Chen, B.; and Vestjens, A. P. A. 1997. Scheduling on identical machines: How good is LPT in an on-line setting? *Oper. Res. Lett.*, 21(4): 165–169.

Cropper, A.; and Dumančić, S. 2022. Inductive logic programming at 30: a new introduction. *Journal of Artificial Intelligence Research*, 74: 765–850.

Cropper, A.; and Morel, R. 2021. Learning programs by learning from failures. *Machine Learning*, 110(4): 801–856.

Garey, M. R.; and Johnson, D. S. 1979. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. New York: W. H. Freeman and Company. ISBN 0716710455.

Gerkey, B. P.; and Matarić, M. J. 2002. Sold!: Auction Methods for Multirobot Coordination. *IEEE Transactions on Robotics Automation*, 18(5): 758.

Gerkey, B. P.; and Matarić, M. J. 2004. A formal analysis and taxonomy of task allocation in multi-robot systems. *The International journal of robotics research*, 23(9): 939–954.

Graham, R. L. 1969. Bounds on multiprocessing timing anomalies. *SIAM J. Appl. Math.*, 17: 416–429.

Graham, R. L.; Lawler, E. L.; Lenstra, J. K.; and Kan, A. R. 1979. Optimization and approximation in deterministic sequencing and scheduling: a survey. In *Annals of discrete mathematics*, volume 5, 287–326. Elsevier.

Korsah, G. A.; Stentz, A.; and Dias, M. B. 2013. A comprehensive taxonomy for multi-robot task allocation. *The International journal of robotics research*, 32(12): 1495–1512.

Pinedo, M. L. 2016. *Scheduling: Theory, Algorithms, and Systems*. Springer, 5 edition.

Stentz, A.; and Dias, M. B. 1999. A free market architecture for coordinating multiple robots. Technical Report CMU-RI-TR-99-42, Carnegie Mellon University, Pittsburgh, PA, USA.

Yan, Z.; Jouandeau, N.; and Cherif, A. A. 2011. Multi-robot decentralized exploration using a tradebased approach. In *Proceedings of ICINCO'11*, 99–105.

Yan, Z.; Jouandeau, N.; and Cherif, A. A. 2013. A Survey and Analysis of Multi-Robot Coordination. *International Journal of Advanced Robotic Systems*, 10(12): 399.